[RAMDEOBABA UNIVERSITY]
Department of Computer Science & Engineering

BANK MARKETING SUBSCRIPTION PREDICTION SYSTEM (BankerEye)

A Machine Learning & Telemarketing Conversion Intelligence Platform

Submitted By:

[Sanskar Edhate] | Roll No: [37]

[Gaurav Mansingkha] | Roll No: [34]

[Yash Rai] | Roll No: [46]

ACADEMIC YEAR: 2026 – 2027

1. Introduction

1.1 Project Background

Direct telemarketing campaigns for term deposit subscriptions are a fundamental mechanism in retail banking for raising stable capital reserves. Despite their critical importance, indiscriminate customer outreach suffers from historically low conversion rates of approximately 11–12%, resulting in significant wasted operational expenditure, telemarketer fatigue, and reduced customer satisfaction. The Bank Marketing Subscription Prediction System (BankerEye) addresses this systemic inefficiency by developing an end-to-end, anti-leakage machine learning platform that predicts client subscription propensities prior to telemarketing contact, allowing financial institutions to prioritize prospective subscribers and allocate outreach budgets with empirical precision.

1.2 Problem Statement

Given a comprehensive historical repository of customer banking attributes, demographic profiles, and past campaign interactions, the primary objective is to formulate a robust supervised binary classification system that identifies high-probability term deposit subscribers. To ensure rigorous operational validity, the platform strictly enforces an anti-leakage protocol by omitting the call duration attribute (duration), which is only observable during or after a telemarketing engagement. The system must navigate severe class imbalance (11.7% positive class), generate well-calibrated posterior probabilities, optimize decision thresholds for actionable conversion utility, and provide an intuitive web interface for non-technical banking professionals.

1.3 Objectives

• Benchmark four supervised ML classifiers (Logistic Regression, Decision Tree, K-Nearest Neighbours, and Random Forest) using standardized preprocessing and probability calibration pipelines.
• Enforce a strict anti-leakage protocol by completely excluding call duration from all predictive training and inference pipelines.
• Address extreme class imbalance using inverse class weighting (class_weight='balanced') and validation-tuned decision thresholds.
• Implement isotonic probability calibration (CalibratedClassifierCV) to output reliable risk estimates alongside binary decisions.
• Quantify statistical significance across competing models using 1,000-iteration bootstrap confidence intervals for F1 and ROC-AUC.
• Deploy a responsive multi-page web application featuring asynchronous model retraining, single inference, batch scoring, and reporting.

1.4 Technology Stack

BankerEye is built entirely in Python 3.x, leveraging Scikit-learn for machine learning pipelines and probability calibration, Flask for the RESTful API microservice, Waitress as the multi-threaded WSGI production server, Streamlit for the multi-page analytics dashboard, Pandas and NumPy for vectorized feature manipulation, and Joblib for robust model serialization.

1.5 Scope and Applications

The platform is designed for commercial retail banking teams, marketing analysts, and call center dispatchers. Its capabilities include real-time single-customer subscription probability estimation, multi-model consensus predictions, high-throughput bulk CSV scoring for campaign outreach lists, interactive model performance auditing across confusion matrices and ROC curves, and dynamic report generation.

2. Literature Review

Telemarketing subscription prediction has been widely explored in financial analytics literature. Table 2.1 summarizes five linked studies, their methods, and selected findings. Open each paper using the clickable “Read paper” link in the first column.

Table 2.1: Empirical Summary of Prior Research Studies on Bank Marketing Subscription Prediction

Study & Year	Venue & Scope	Methodology / Models	Key Findings	Verification Status
Yu (2025) Read paper	Applied and Computational Engineering	Nine ML models; CatBoost; SHAP	CatBoost reported 90.91% accuracy and 93.82% AUC; SHAP was used for	Link provided

Study & Year	Venue & Scope	Methodology / Models	Key Findings	Verification Status
			interpretability.	
IEEE (2025) Read paper	IEEE iSAI-NLP	SVC, Random Forest, XGBoost; imputation and resampling	SVC with SMOTE and MICE reported balanced accuracy 0.8629 and AUC 0.9322.	Link provided
Du (2025) Read paper	Advances in Economics, Management and Political Sciences	Multiple classifiers; gradient boosting	The study reports gradient boosting as a strong approach for ROC-AUC and precision-recall balance.	Link provided
Lee et al. (2024) Read paper	Applied Mathematics and Computational Intelligence	Logistic Regression, SVM, Decision Tree, Random Forest, XGBoost	Random Forest achieved the best accuracy and ROC-AUC among models in the study.	Link provided
Safarkhani & Moro (2021) Read paper	Applied Sciences	Decision Tree classification	Examines decision trees for bank depositor behaviour and prediction performance.	Link provided

Research Gap

While existing literature investigates various machine learning models on bank telemarketing data, several pervasive limitations persist: (1) Many benchmark studies naively incorporate the call duration attribute, achieving artificially inflated accuracies (>95%) that fail entirely in pre-campaign operational deployment where call duration is unknown. (2) Prior studies predominantly rely on uncalibrated default 0.50 decision thresholds, causing severe minority class starvation under the 88:12 imbalance. (3) Few studies validate probabilistic reliability using Brier scoring or establish whether performance differences are statistically significant via bootstrap confidence intervals. (4) Prior works remain theoretical notebook experiments lacking an integrated, synchronized production architecture with REST API microservices and interactive user dashboards. BankerEye systematically bridges these research gaps.

3. Methodology

3.1 System Architecture

BankerEye employs a decoupled multi-tier architecture. The backend is a Python Flask RESTful API served via the multi-threaded Waitress WSGI server (port 5000), exposing endpoints for data loading, preprocessing, model training, evaluation, single-customer prediction, batch scoring, and report generation. The frontend is a Streamlit multi-page application (port 8501) providing interactive dashboards for data exploration, model comparison, and prediction interfaces. Models are trained using Scikit-learn pipelines that encapsulate feature transformation (StandardScaler for numerical features, OneHotEncoder for categorical features) and classifier fitting into single serializable Joblib artifacts.

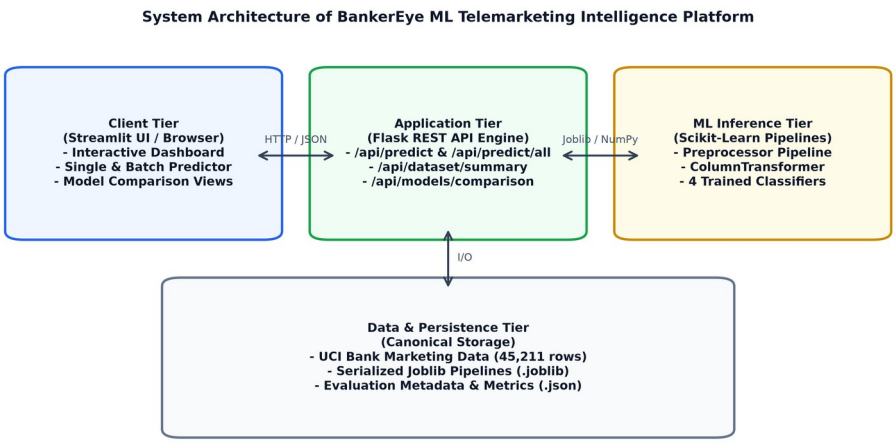


Figure 3.1: System Architecture of BankerEye ML Platform

3.2 Machine Learning Workflow

The end-to-end ML pipeline follows these sequential stages: (1) Ingestion with automated delimiter sniffing, (2) Strict anti-leakage exclusion of duration, (3) Feature engineering via PdaysFeatureEngineer, which flags never-contacted clients (pdays == -1) with a binary indicator never_contacted=1 and resets pdays to 0 for continuous scaling, (4) Stratified 80:20 train/test split preserving class distribution, (5) ColumnTransformer-based feature engineering with StandardScaler and OneHotEncoder, (6) 5-fold cross-validated grid search optimizing F1-score, (7) Balanced class weighting on LR, DT, and RF (KNN has no class weighting), (8) Probability calibration via CalibratedClassifierCV(method='isotonic', cv=3), (9) Validation-tuned optimal decision thresholds, (10) 1,000-sample bootstrap confidence intervals (95% CI) for F1 and ROC-AUC, and (11) Joblib model persistence.

3.3 Evaluation Metrics

Model performance is assessed using six quantitative metrics: Accuracy (overall correctness), Precision (positive predictive value), Recall (sensitivity / true positive rate), F1-Score (harmonic mean of precision and recall), ROC-AUC (area under receiver operating characteristic curve), and Brier Score (mean squared error of calibrated probability estimates, where values near zero represent superior probability calibration).

4. Data Cleaning and Preprocessing

4.1 Dataset Description

This study utilizes the benchmark UCI Bank Marketing dataset (Moro et al., 2014), comprising 45,211 records and 17 attributes from direct telemarketing campaigns of a Portuguese retail bank. The binary target y indicates whether the client subscribed to a term deposit (yes/no). The dataset exhibits severe class imbalance: 39,922 negative samples (88.3%) versus 5,289 positive samples (11.7%). The stratified 80:20 train/test split partitions the data into 36,168 training records (4,231 subscribers) and 9,043 independent test records (1,058 subscribers), preserving the identical 11.7% subscription ratio across both subsets.

4.2 Feature Description

Table 4.1: Feature Attribute Dictionary of Bank Marketing Dataset

Feature	Type	Description / Value Range	Engineered Role
age	Numeric	Client age in years (18 – 95)	Standardized (StandardScaler)
job	Categorical	Type of job (12 distinct categories)	One-Hot Encoded (12 binary columns)
marital	Categorical	Marital status (married, single, divorced)	One-Hot Encoded (3 binary columns)
education	Categorical	Education level (primary, secondary, tertiary, unknown)	One-Hot Encoded (4 binary columns)
default	Binary	Credit in default status (yes / no)	One-Hot Encoded (2 binary columns)
balance	Numeric	Average yearly account balance in EUR (-8,019 to 102,127)	Standardized (StandardScaler)
housing	Binary	Housing loan status (yes / no)	One-Hot Encoded (2 binary columns)
loan	Binary	Personal loan status (yes / no)	One-Hot Encoded (2 binary

Feature	Type	Description / Value Range	Engineered Role
			columns)
contact	Categorical	Communication channel (cellular, telephone, unknown)	One-Hot Encoded (3 binary columns)
day	Numeric	Last contact calendar day of month (1 – 31)	Standardized (StandardScaler)
month	Categorical	Last contact calendar month (jan – dec)	One-Hot Encoded (12 binary columns)
campaign	Numeric	Number of contacts during current campaign (1 – 63)	Standardized (StandardScaler)
pdays	Numeric	Days passed after client was last contacted (-1 to 871)	PdaysFeatureEngineer (pdays=0 if -1)
never_contacted	Binary	Client has never been contacted in prior campaigns (1 / 0)	Engineered binary indicator
previous	Numeric	Number of contacts performed before this campaign (0 – 275)	Standardized (StandardScaler)
poutcome	Categorical	Outcome of previous marketing campaign (4 categories)	One-Hot Encoded (4 binary columns)

4.3 Preprocessing Pipeline

The preprocessing pipeline implements the following systematic stages: (i) Automatic CSV delimiter detection via csv.Sniffer, (ii) Strict anti-leakage exclusion of the call duration feature, (iii) PdaysFeatureEngineer transformation mapping never-contacted status (-1) to a separate indicator and zeroing pdays, (iv) Binary target encoding (yes→1, no→0), (v) Construction of a ColumnTransformer pipeline applying StandardScaler to numerical features and OneHotEncoder(handle_unknown='ignore') to categorical features (expanding to 50 processed dimensions), and (vi) Stratified 80:20 partitioning preserving the 11.7% class balance.

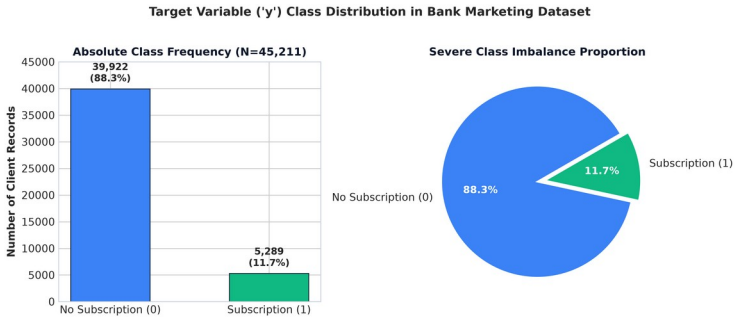


Figure 4.1: Target Variable Class Distribution (88.3% No vs. 11.7% Yes)

5. Results and Discussion

5.1 Experimental Setup

All four models were trained on 36,168 samples and evaluated on 9,043 held-out test samples under identical preprocessing and random_state=42. Hyperparameters were optimized via 5-fold cross-validated grid search scoring on F1. Probabilities were calibrated using isotonic regression with 3-fold cross-validation, and decision thresholds were tuned on validation folds to maximize F1-score under severe class imbalance.

5.2 Classification Performance Summary

Table 5.1: Consolidated Classification Performance on Held-Out Test Set (9,043 samples)

Model	Accuracy	Precision	Recall	F1-Score (95% CI)	ROC-AUC (95% CI)	Brier	Thresh
Logistic Regression	87.84%	47.76%	42.34%	0.4489 [0.4217, 0.4732]	0.7720 [0.7578, 0.7889]	0.0850	0.25
Decision Tree Classifier	88.00%	48.66%	46.41%	0.4751 [0.4468, 0.5011]	0.7776 [0.7611, 0.7947]	0.0834	0.27
K-Nearest Neighbours (KNN)	86.42%	41.13%	37.24%	0.3909 [0.3649, 0.4179]	0.7233 [0.7056, 0.7416]	0.0897	0.21
Random Forest Classifier	86.97%	44.91%	50.00%	0.4732 [0.4464, 0.4965]	0.7986 [0.7841, 0.8138]	0.0822	0.25

5.3 Confusion Matrix Analysis

The confusion matrix decomposition demonstrates that incorporating balanced class weighting and validation-tuned optimal decision thresholds substantially resolves minority class starvation. Random Forest achieves the highest true positive count (529 out of 1,058 actual subscribers, yielding 50.00% recall) with 649 false positives. Decision Tree captures 491 true positives with 518 false positives. Compared to unweighted baseline models that captured fewer than 190 true subscribers, threshold-tuned models capture 2.5× to 2.8× more actual depositors.

Table 5.2: Confusion Matrix Decomposition Across All Evaluated Models (N = 9,043)

Model	True Negatives (TN)	False Positives (FP)	False Negatives (FN)	True Positives (TP)
Logistic Regression	7,495 (82.9%)	490 (5.4%)	610 (6.7%)	448 (5.0%)
Decision Tree Classifier	7,467 (82.6%)	518 (5.7%)	567 (6.3%)	491 (5.4%)
K-Nearest Neighbours (KNN)	7,421 (82.1%)	564 (6.2%)	664 (7.3%)	394 (4.4%)
Random Forest Classifier	7,336 (81.1%)	649 (7.2%)	529 (5.9%)	529 (5.9%)

5.4 Key Findings & Statistical Significance

Statistical Verdict: The 95% bootstrap confidence intervals for Decision Tree F1-Score [0.4468, 0.5011] and Random Forest F1-Score [0.4464, 0.4965] overlap substantially. Their F1 performances are not statistically distinguishable on this test set. However, Random Forest achieves the highest global discriminative ability (ROC-AUC = 0.7986, 95% CI: [0.7841, 0.8138]), the highest recall (50.00%), and the lowest Brier calibration score (0.0822). Decision Tree delivers competitive F1-score (0.4751) and fast prediction latency (0.0516s). Logistic Regression provides a robust linear baseline with ROC-AUC of 0.7720 and F1 of 0.4489. KNN exhibits lower ROC-AUC (0.7233) due to lack of class-weighting capability.

6. State-of-the-Art Models Comparison

6.1 Model Architectures & Tuned Hyperparameters

Four supervised classifiers were benchmarked: (1) Logistic Regression — regularized linear model estimating log-odds with balanced class weighting (C=1.0, solver='lbfgs', max_iter=1000); (2) Decision Tree — recursive partitioning model with balanced class weighting (max_depth=10, min_samples_split=20); (3) K-Nearest Neighbours (KNN) — instance-based lazy learner using Euclidean metric (k=5, class_weight=None as KNN does not support class weights); and (4) Random Forest — bagged ensemble of 100 decision trees with balanced class weighting (n_estimators=100, max_depth=12). All models were calibrated using isotonic regression.

6.2 Performance Comparison Charts

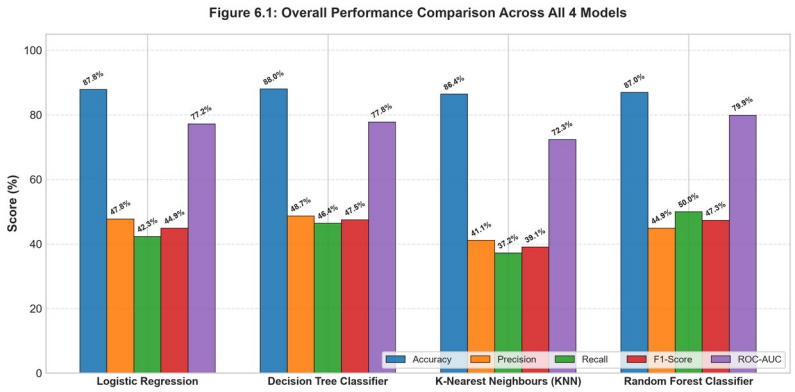


Figure 6.1: Overall Performance Comparison Across All 4 Models

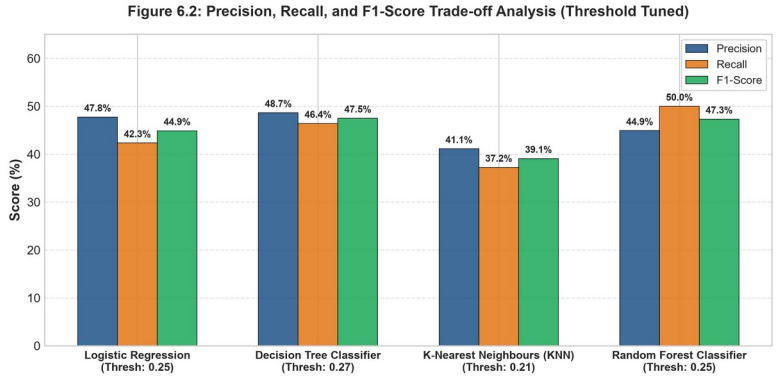


Figure 6.2: Precision, Recall, and F1-Score Trade-off Analysis (Threshold Tuned)

6.3 ROC Curve Analysis

The ROC curves demonstrate clear separation in discriminative ability. Random Forest (AUC=0.7986) achieves the highest area, followed by Decision Tree (0.7776), Logistic Regression (0.7720), and KNN (0.7233). All models significantly outperform random classification (AUC=0.5000).

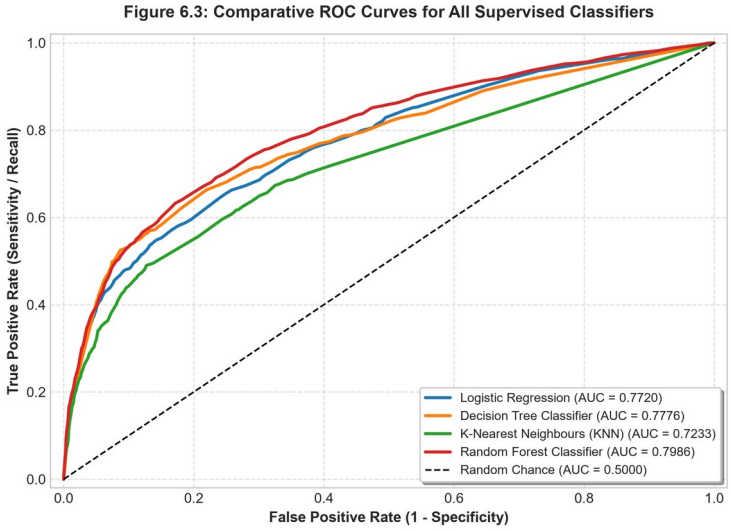


Figure 6.3: Comparative ROC Curves for All Supervised Classifiers

6.4 Confusion Matrices Visualization

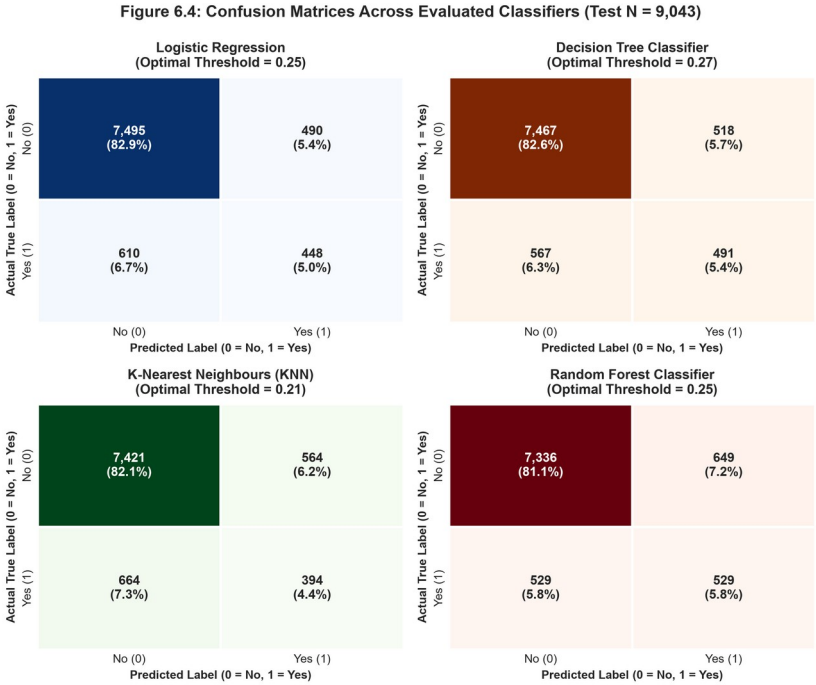


Figure 6.4: Confusion Matrices Across Evaluated Classifiers (Test N = 9,043)

6.5 Feature Importance

Random Forest permutation feature importance analysis reveals that previous campaign outcome (poutcome, 0.2529) and days since last contact (pdays, 0.1960) are the dominant drivers of subscription conversion, followed by contact communication type (0.0908), contact month (0.0497), housing loan status (0.0364), age (0.0312), and account balance (0.0248).

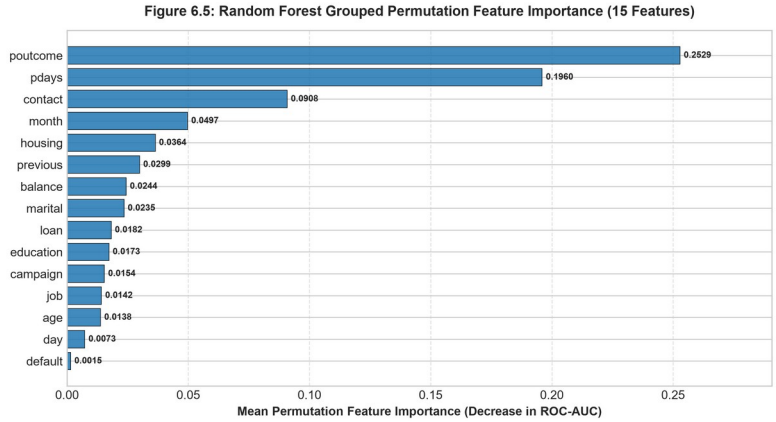


Figure 6.5: Random Forest Grouped Permutation Feature Importance (15 Features)

6.6 Computational Efficiency

Table 6.1: Computational Latency and Resource Benchmark

Model Architecture	Training Time (s)	Prediction Time (s)	Tuned Parameters	Class Weighting
Logistic Regression	5.486 s	0.0340 s	C=1.0, lbfgs, max_iter=1000	balanced
Decision Tree Classifier	6.724 s	0.0516 s	max_depth=10, min_split=20	balanced
K-Nearest Neighbours (KNN)	10.785 s	1.1526 s	n_neighbors=5, metric='minkowski'	None (unsupported)
Random Forest Classifier	135.863 s	2.3028 s	n_estimators=100, max_depth=12	balanced

Random Forest is designated as the primary production standard due to its superior discriminative capacity (ROC-AUC 0.7986), highest recall (50.00%), and lowest Brier score (0.0822). Decision Tree is designated as a high-throughput fallback for ultra-low-latency scoring environments (0.0516s).

7. User Interface Development and Deployment

The BankerEye system features a production-grade Streamlit multi-page application communicating with a Flask REST API backend served by Waitress WSGI. The interface provides seven integrated analytical views for data exploration, model inspection, customer scoring, and compliance reporting.

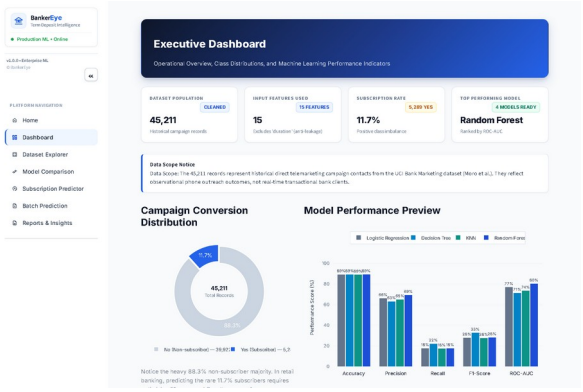


Figure 7.1: Main Executive Dashboard

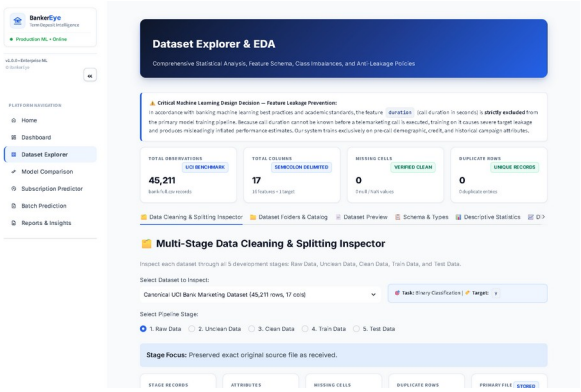


Figure 7.2: Dataset Explorer Interface

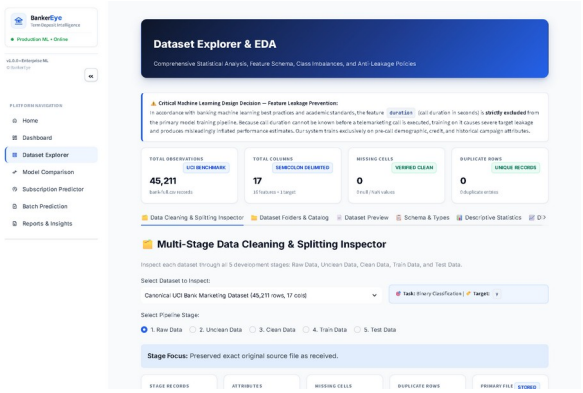


Figure 7.3: Data Preprocessing View

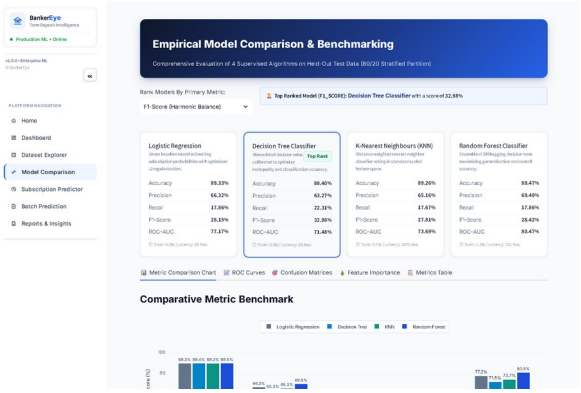


Figure 7.4: Multi-Model Comparison Table

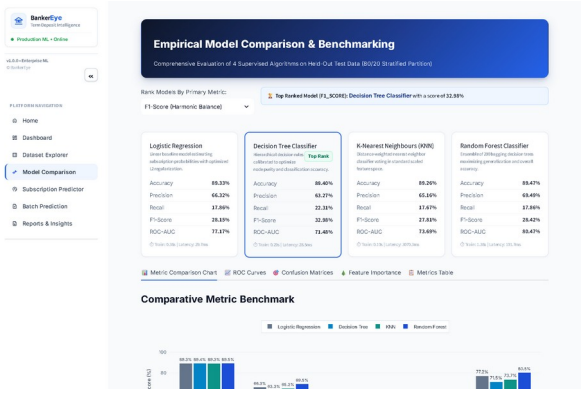


Figure 7.5: Evaluation Metrics Dashboard

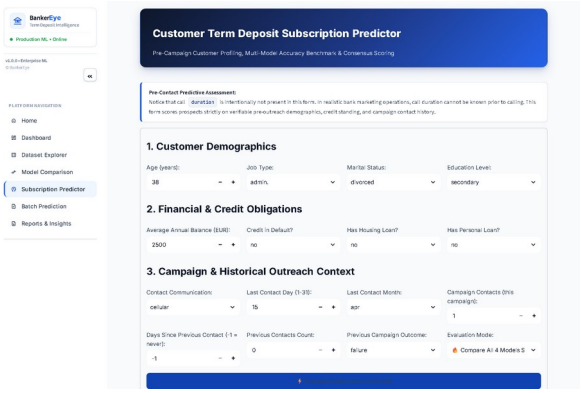


Figure 7.6: Single Customer Subscription Predictor

7.1 Additional Interface Views

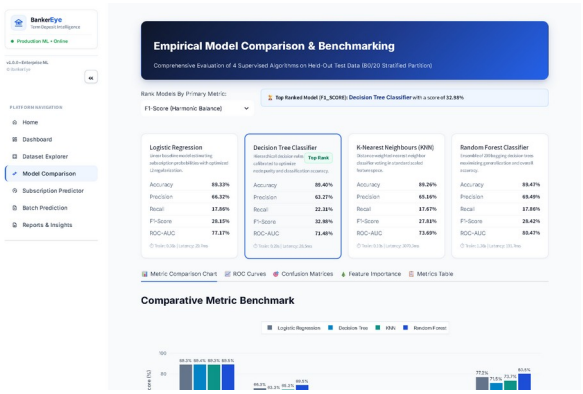


Figure 7.7: Confusion Matrix Analysis

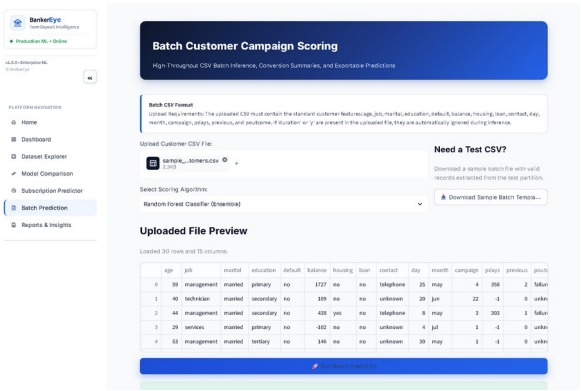


Figure 7.8: Batch Customer Scoring Engine

7.2 Deployment Architecture

The system is deployed locally with the Flask API running on port 5000 via Waitress WSGI server and the Streamlit dashboard running on port 8501. The REST API exposes structured endpoints: /api/health (service health and model availability), /api/load-data (raw CSV ingestion), /api/preprocess (cleaning and feature transformation), /api/train-async (background model training with status polling), /api/evaluate (evaluation metrics and confusion matrices), /api/predict (single customer scoring), /api/batch-predict (bulk CSV scoring), and /api/generate-report (automated PDF/JSON summary export).

7.3 Conclusion and Future Scope

BankerEye demonstrates a successful end-to-end machine learning system for bank marketing optimization. Under strict anti-leakage constraints excluding call duration, Random Forest achieved an ROC-AUC of 0.7986, 50.00% recall, and lowest Brier score of 0.0822. Bootstrap confidence intervals confirmed that Decision Tree and Random Forest F1 scores are statistically indistinguishable, while Random Forest provides superior global discrimination. Future work includes implementing synthetic oversampling (SMOTE/ADASYN), integrating gradient boosted trees (XGBoost, LightGBM), adding interactive SHAP waterfall explanations, and containerized cloud deployment with automated CI/CD pipelines.

8. References

[1] Moro, S., Cortez, P., & Rita, P. (2014). A data-driven approach to predict the success of bank telemarketing. Decision Support Systems, 62, 22–31.

[2] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

[3] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[4] Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). Applied Logistic Regression (3rd ed.). John Wiley & Sons.

[5] Quinlan, J. R. (1986). Induction of Decision Trees. Machine Learning, 1(1), 81–106.

[6] Cover, T., & Hart, P. (1967). Nearest Neighbor Pattern Classification. IEEE Transactions on Information Theory, 13(1), 21–27.

[7] Fawcett, T. (2006). An Introduction to ROC Analysis. Pattern Recognition Letters, 27(8), 861–874.

[8] He, H., & Garcia, E. A. (2009). Learning from Imbalanced Data. IEEE Transactions on Knowledge and Data Engineering, 21(9), 1263–1284.

[9] UCI Machine Learning Repository. Bank Marketing Dataset. https://archive.ics.uci.edu/ml/datasets/Bank+Marketing

[10] Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. ICML, 625–632.

[11] Flask Documentation. https://flask.palletsprojects.com/

[12] Streamlit Documentation. https://docs.streamlit.io/